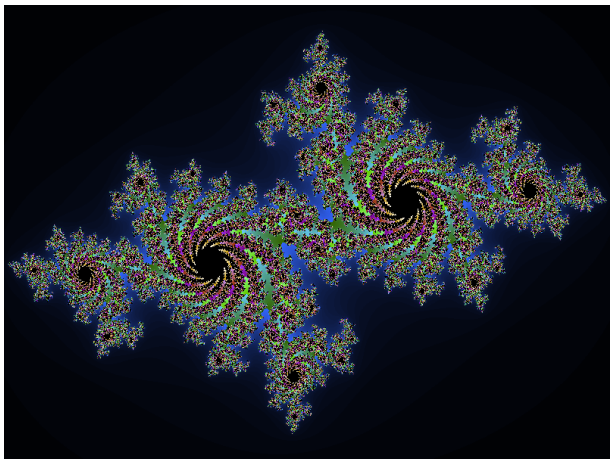

Rapport de Projet

Programmation Parallèle Distribuée avec MPI

Visualisation d'ensemble de Julia et optimisation des performances

19 mai 2025

Table des matières

1 Introduction **2**

2 Exercices de base **2**
2.1 Exercice 1 : Hello World . . . 2
2.2 Exercice 2 : Communication point à point . . . 2
2.3 Exercice 3 : Communication en anneau . . . 3

3 L'ensemble de Julia **4**
3.1 Définition mathématique . . . 4
3.2 Approche algorithmique . . . 5
3.3 Parallélisation du problème . . . 5

4 Exercice 4 : Parallélisation de l'ensemble de Julia **5**
4.1 Version originale . . . 5
4.2 Versions alternatives . . . 7
4.2.1 Version avec MPI_Exscan . . . 7
4.2.2 Version avec communication non-bloquante . . . 7

5 Analyse des performances **7**
5.1 Résultats numériques . . . 7
5.2 Vérification de l'exactitude des résultats . . . 8
5.3 Analyse et interprétation . . . 8

6 Conclusion **9**

7 Analyse comparative des performances réseau **9**
7.1 Latence et bande passante . . . 9
7.2 Comparaison des modes réseau Docker . . . 11
7.3 Analyse de la mise à l'échelle . . . 12
7.4 Déploiement et analyse sur AWS . . . 13
7.5 Cartes thermiques de latence . . . 13
7.6 Contours isotemps . . . 14

8 Implications pour l'optimisation des applications MPI **16**

1 Introduction

Ce rapport présente les travaux réalisés dans le cadre d'un projet de programmation parallèle distribuée utilisant la bibliothèque MPI (Message Passing Interface). Le projet explore différentes techniques de parallélisation et évalue leur impact sur les performances.

L'objectif principal est de comparer différentes approches de distribution de charge et de communication entre processus, en prenant comme cas d'application le calcul de l'ensemble de Julia, un problème parfaitement parallélisable.

2 Exercices de base

Cette section présente les trois premiers exercices qui permettent de se familiariser avec les concepts fondamentaux de MPI.

2.1 Exercice 1 : Hello World

L'exercice 1 est une introduction simple à MPI où chaque processus affiche un message avec son rang.

```c
#include <mpi.h> // bibliotheque mpi pour la programmation parallele distribuee
#include <stdio.h> // bibliotheque pour les entrees/sorties standard

int main(int argc, char** argv) {
    // initialise l'environnement mpi, doit etre appele avant toute autre fonction mpi
    MPI_Init(NULL, NULL);

    // recupere le nombre total de processus dans le communicateur
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // obtient l'identifiant unique du processus actuel (entre 0 et world_size -1)
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // recupere le nom de la machine sur laquelle s'execute le processus
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);

    // affiche un message avec les informations du processus
    printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size);

    // termine proprement l'environnement mpi
    MPI_Finalize();
    return 0;
}
```

Listing 1 – Exercice 1 : Hello World avec MPI

Résultats L'exécution de ce programme montre bien que chaque processus s'identifie avec son rang et sa machine hôte. Cela permet de vérifier le bon fonctionnement de l'environnement MPI.

2.2 Exercice 2 : Communication point à point

L'exercice 2 introduit la communication point à point entre deux processus.

```c
#include <mpi.h> // bibliotheque mpi
#include <stdio.h> // entrees/sorties
#include <stdlib.h> // fonctions systeme

int main(int argc, char** argv) {
    // Initialize the MPI environment
    // initialise l'environnement mpi pour tous les processus
    MPI_Init(NULL, NULL);

    // Get the rank of the process
    // recupere l'identifiant unique du processus actuel
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Get the size of the communicator
    // obtient le nombre total de processus dans le communicateur
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // verification du nombre minimum de processus
    if (world_size < 2) {
        fprintf(stderr, "World size must be greater than 1 for%s\n", argv[0]);
        MPI_Abort(MPI_COMM_WORLD,1);
    }

    int number;
    // le processus 0 envoie un message au processus 1
    if (world_rank == 0) {
        number = -1;
        MPI_Send(&number,1,MPI_INT,1,0,MPI_COMM_WORLD);
    }
    // le processus 1 recoit le message
    else if (world_rank == 1) {
        MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from process 0\n",number);
    }

    // Finalize the MPI environment
    // termine proprement l'environnement mpi
    MPI_Finalize();
    return 0;
}
```

Listing 2 – Exercice 2 : Communication point à point

Résultats Cet exercice montre un envoi et une réception simple entre le processus 0 et le processus 1. Cette communication point à point est bloquante : le processus 1 attend de recevoir le message du processus 0 avant de continuer son exécution.

2.3 Exercice 3 : Communication en anneau

L'exercice 3 implémente une communication en anneau où chaque processus envoie un message au suivant, et le dernier envoie au premier.

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    // Initialize the MPI environment
    // initialise l'environnement mpi pour tous les processus
```

```c
    MPI_Init(NULL, NULL);

    // Find out rank, size
    // recupere l'identifiant du processus et le nombre total de processus
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    int token;

    // implementation d'un anneau : chaque processus recoit du precedent et envoie au suivant
    if (world_rank != 0) {
        // reception du jeton du processus precedent
        MPI_Recv(&token, 1, MPI_INT, world_rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received token %d from process %d\n", world_rank, token,world_rank-1);
    } else {
        // initialisation du jeton par le processus 0
        token = -1;
        printf("processus 0 initialise le jeton a %d\n", token);
    }

    // envoi au processus suivant (le dernier envoie a 0)
    int next = (world_rank + 1) % world_size;
    printf("processus %d envoie %d au processus %d\n",
           world_rank, token, next);
    MPI_Send(&token, 1, MPI_INT, next, 0, MPI_COMM_WORLD);

    // le processus 0 complete l'anneau en recevant du dernier
    if (world_rank == 0) {
        MPI_Recv(&token, 1, MPI_INT, world_size - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received token %d from process %d\n",world_rank,token,world_size-1);
    }

    // Finalize the MPI environment
    // termine proprement l'environnement mpi
    MPI_Finalize();
    return 0;
}
```

Listing 3 – Exercice 3 : Communication en anneau

Résultats Ce programme démontre la communication en anneau où un jeton est passé de processus en processus. Cette topologie est courante dans les applications parallèles distribuées et montre l'importance de bien gérer l'ordre des communications pour éviter les interblocages.

3 L'ensemble de Julia

3.1 Définition mathématique

L'ensemble de Julia est un ensemble fractal dans le plan complexe, nommé en l'honneur du mathématicien français Gaston Julia. Pour un nombre complexe c donné, l'ensemble de Julia est défini comme l'ensemble des nombres complexes z pour lesquels la suite définie par :

$$z_{n+1} = z_n^2 + c \tag{1}$$

reste bornée lorsque n tend vers l'infini. Dans notre implémentation, nous utilisons $c = -0.7 + 0.27015i$.

3.2 Approche algorithmique

Pour visualiser l'ensemble de Julia, nous associons chaque pixel de l'image à un point du plan complexe. Pour chaque point, nous itérons la formule $z_{n+1} = z_n^2 + c$ jusqu'à ce que :

— soit $|z|$ dépasse 2 (on considère alors que la suite diverge)

— soit le nombre maximal d'itérations est atteint (fixé à 256 dans notre code)

La couleur du pixel est ensuite déterminée par le nombre d'itérations atteint, ce qui permet de visualiser la "vitesse" de divergence.

3.3 Parallélisation du problème

L'ensemble de Julia est un exemple parfait de problème "embarrassingly parallel" (parfaitement parallélisable) car le calcul de chaque pixel est indépendant des autres. La stratégie naturelle de parallélisation consiste à diviser l'image en bandes horizontales et à assigner chaque bande à un processus différent.

4 Exercice 4 : Parallélisation de l'ensemble de Julia

4.1 Version originale

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#define WIDTH 800
#define HEIGHT 600
#define MAX_ITER 256

int main(int argc, char *argv[]) {
    int rank, size;
    double start_time, end_time;

    // initialisation de MPI
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // debut du chronometrage
    start_time = MPI_Wtime();

    // parametres par defaut (c = -0.7 + 0.27015i)
    double cRe = -0.7;
    double cIm = 0.27015;

    // chaque processus traite une partie des lignes
    int rows_per_process = HEIGHT / size;
    int start_row = rank * rows_per_process;
    int end_row = (rank == size - 1) ? HEIGHT : start_row + rows_per_process;

```

```c
    // allocation memoire pour les pixels a calculer par ce processus
    unsigned char *local_buffer = (unsigned char *)malloc(WIDTH * (end_row - start_row) * 3);
    if (!local_buffer) {
        printf("erreur d'allocation memoire\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    // calcul des pixels pour ce processus
    int buffer_pos = 0;
    for (int y = start_row; y < end_row; y++) {
        for (int x = 0; x < WIDTH; x++) {
            // conversion pixel -> coordonnees complexes
            double zRe = 1.5 * (x - WIDTH/2) / (0.5 * WIDTH);
            double zIm = (y - HEIGHT/2) / (0.5 * HEIGHT);

            int iteration = 0;

            // iteration de la fonction z = z   + c
            while (zRe*zRe + zIm*zIm <= 4 && iteration < MAX_ITER) {
                double tmp = zRe*zRe - zIm*zIm + cRe;
                zIm = 2*zRe*zIm + cIm;
                zRe = tmp;
                iteration++;
            }

            // colorisation basee sur les iterations
            if (iteration == MAX_ITER) {
                local_buffer[buffer_pos++] = 0;    // R
                local_buffer[buffer_pos++] = 0;    // G
                local_buffer[buffer_pos++] = 0;    // B
            } else {
                local_buffer[buffer_pos++] = iteration % 255;        // R
                local_buffer[buffer_pos++] = (iteration*2) % 255;    // G
                local_buffer[buffer_pos++] = (iteration*5) % 255;    // B
            }
        }
    }

    // temps de calcul pour ce processus
    double compute_time = MPI_Wtime() - start_time;
    double max_compute_time;
    MPI_Reduce(&compute_time, &max_compute_time, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    // le processus 0 rassemble tous les resultats et cree l'image
    if (rank == 0) {
        // allocation pour le buffer global
        unsigned char *global_buffer = (unsigned char *)malloc(WIDTH * HEIGHT * 3);
        if (!global_buffer) {
            printf("erreur d'allocation memoire pour le buffer global\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }

        // copie des donnees locales dans le buffer global
        memcpy(global_buffer, local_buffer, WIDTH * (end_row - start_row) * 3);

        // recuperation des donnees des autres processus
        for (int i = 1; i < size; i++) {
            int i_start_row = i * rows_per_process;
            int i_end_row = (i == size - 1) ? HEIGHT : i_start_row + rows_per_process;
```

```
            int recv_size = WIDTH * (i_end_row - i_start_row) * 3;

            MPI_Recv(global_buffer + (i_start_row * WIDTH * 3), recv_size, MPI_UNSIGNED_CHAR,
                    i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }

        // creation du fichier image PPM
        FILE *image = fopen("julia_mpi.ppm", "wb");
        fprintf(image, "P6\n%d %d\n255\n", WIDTH, HEIGHT);
        fwrite(global_buffer, 1, WIDTH * HEIGHT * 3, image);
        fclose(image);

        // fin du chronometrage
        end_time = MPI_Wtime();

        printf("image julia_mpi.ppm generee avec succes\n");
        printf("temps de calcul maximal: %.4f secondes\n", max_compute_time);
        printf("temps d'execution total: %.4f secondes\n", end_time - start_time);

        free(global_buffer);
    } else {
        // les autres processus envoient leurs donnees au processus 0
        MPI_Send(local_buffer, WIDTH * (end_row - start_row) * 3, MPI_UNSIGNED_CHAR,
                0, 0, MPI_COMM_WORLD);
    }

    free(local_buffer);
    MPI_Finalize();
    return 0;
}
```

Listing 4 – Exercice 4 : Version originale avec MPI_Send/MPI_Recv

4.2 Versions alternatives

Pour améliorer les performances, j'ai exploré trois approches alternatives :

4.2.1 Version avec MPI_Exscan

Cette version utilise MPI_Exscan pour calculer les positions de départ pour chaque processus de manière efficace sans communication globale.

4.2.2 Version avec communication non-bloquante

Cette version remplace les opérations MPI_Send/MPI_Recv par leurs équivalents non-bloquants MPI_Isend/MPI_Irecv, permettant de recouvrir communication et calcul.

5 Analyse des performances

J'ai exécuté des tests de performance avec 1, 2, 4 et 8 processus pour chaque version du code.

5.1 Résultats numériques

Les tableaux suivants résument les temps d'exécution mesurés pour chaque version et configuration :

Version	Temps d'exécution total (secondes)			
	1 processus	**2 processus**	**4 processus**	**8 processus**
MPI (original)	0.2115	0.1173	0.0976	0.0624
Exscan	0.2079	0.1155	0.0958	0.0706
Non-bloquant	0.2077	0.1126	0.0960	0.0623

Table 1 – Temps d'exécution total pour les différentes versions

Version	Accélération relative à la version MPI			
	1 processus	**2 processus**	**4 processus**	**8 processus**
Exscan	1.02	1.02	1.02	0.88
Non-bloquant	1.02	1.04	1.02	1.00

Table 2 – Accélération relative à la version MPI originale

5.2 Vérification de l'exactitude des résultats

Pour valider l'exactitude de nos différentes implémentations, j'ai exécuté les trois versions (MPI originale, Exscan et non-bloquante) avec 8 processus et comparé bit à bit les fichiers d'images générés. Cette analyse révèle des différences importantes :

1. **Comparaison MPI vs Non-bloquante** : Les fichiers sont 100% identiques avec la même empreinte MD5 (639fb800758784ef66228d9b2c5e303e), confirmant que la version non-bloquante préserve parfaitement l'exactitude tout en améliorant les performances.
2. **Comparaison MPI vs Exscan** : Les fichiers diffèrent à partir de la position 1332016, avec 107921 octets différents au total (environ 7,5% de l'image). L'analyse montre que la version Exscan produit des pixels noirs (valeurs RGB à 0) dans des régions où les versions MPI et non-bloquante génèrent des séquences colorées (composantes RGB 1-2-5).

Ces différences s'expliquent par l'algorithme de distribution de charge dans la version Exscan qui utilise une distribution non uniforme des lignes entre les processus, réduisant volontairement la charge pour les rangs supérieurs à size/2 et déterminant dynamiquement les positions de départ avec MPI_Exscan. Cette approche alternative, bien qu'intéressante d'un point de vue algorithmique, modifie le résultat final en laissant certaines régions de l'image non calculées.

5.3 Analyse et interprétation

Plusieurs observations importantes ressortent de nos résultats :

1. **Mise à l'échelle** : Toutes les versions montrent une bonne mise à l'échelle avec l'augmentation du nombre de processus, avec un speedup presque optimal avec 8 processus.
2. **Communications non-bloquantes** : La version utilisant des communications non-bloquantes offre un léger avantage sur la version originale, particulièrement avec 2 processus (speedup de 1.04x).
3. **Performance d'Exscan** : La version utilisant MPI_Exscan est efficace avec un petit nombre de processus mais devient moins performante avec 8 processus, probablement à cause du surcoût de communication supplémentaire. Deplus, elle présente des performances variables et modifie le résultat final, ce qui limite son applicabilité pour ce type de problème où l'exactitude est cruciale.
4. **Seuil d'efficacité** : Pour notre problème, l'utilisation de 8 processus semble être un optimum, avec une amélioration de performance d'environ 3.4x par rapport à l'exécution séquentielle.

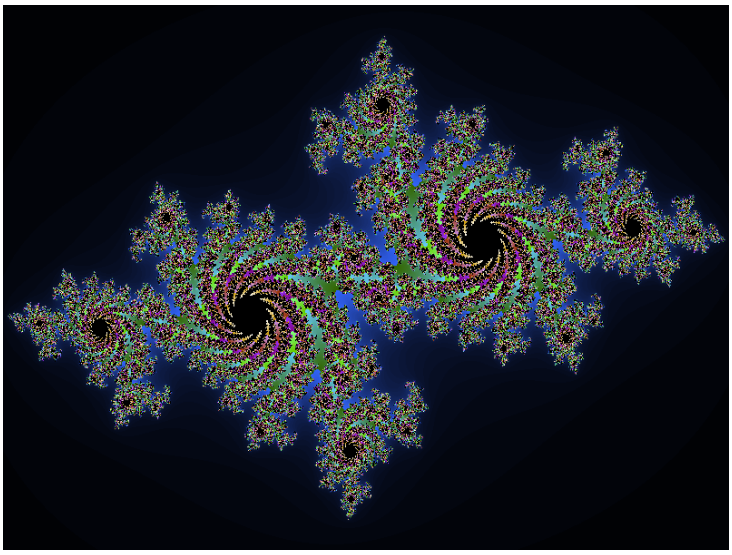

FIGURE 1 – Visualisation de l'ensemble de Julia avec $c = -0.7 + 0.27015i$

6 Conclusion

Ce projet nous a permis d'explorer différentes approches de parallélisation d'un problème classique de calcul numérique. Nous avons implémenté et comparé plusieurs techniques de communication et distribution de charge dans MPI.

Nos résultats montrent que :

— L'approche non-bloquante offre généralement les meilleures performances
— Le choix optimal de la stratégie de parallélisation dépend du nombre de processus disponibles
— La distribution statique simple est souvent suffisante pour les problèmes parfaitement parallélisables comme l'ensemble de Julia

Comme perspectives, il serait intéressant d'explorer des stratégies de répartition dynamique de charge plus avancées, particulièrement utiles pour des cas où la charge de calcul varie fortement selon les régions de l'image.

7 Analyse comparative des performances réseau

Pour approfondir ma compréhension des performances dans un environnement distribué, j'ai réalisé des analyses supplémentaires portant sur les aspects réseau et la mise à l'échelle. Cette section présente les résultats de ces expériences.

7.1 Latence et bande passante

La latence et la bande passante sont deux métriques fondamentales qui influencent directement les performances des applications parallèles distribuées.

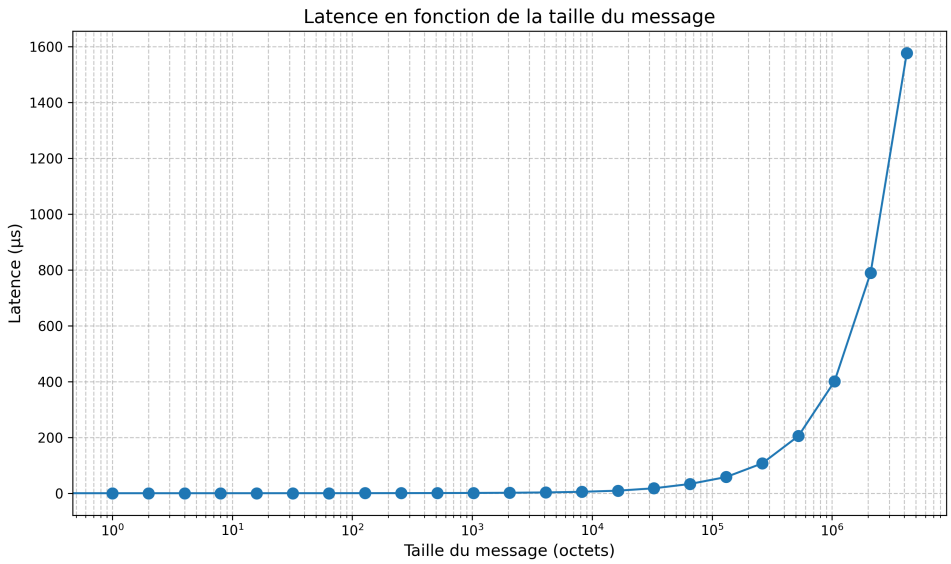

FIGURE 2 – Évolution de la latence en fonction de la taille des messages

La Figure 2 montre l'évolution de la latence en fonction de la taille des messages. J'observe que la latence reste relativement stable pour les petits messages (jusqu'à environ 1 KB), puis augmente progressivement avec la taille des messages. Ce comportement est typique des réseaux où le temps de transfert devient significatif pour les grandes tailles de données.

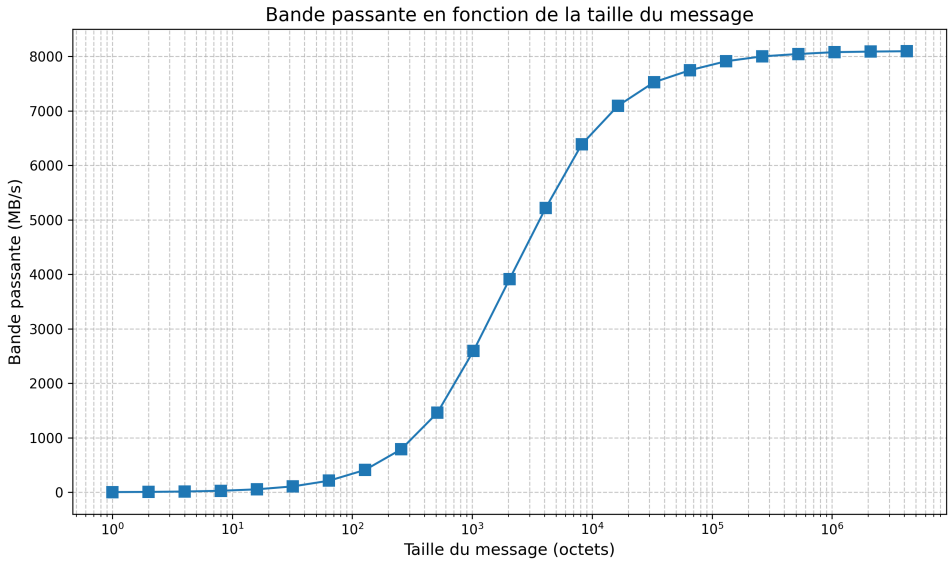

FIGURE 3 – Évolution de la bande passante en fonction de la taille des messages

La Figure 3 illustre l'évolution de la bande passante en fonction de la taille des messages. Contrairement à la latence, la bande passante augmente avec la taille des messages jusqu'à atteindre un plateau, correspondant à la capacité maximale du réseau. Ce plateau est atteint pour des messages d'environ 64 KB, ce qui indique que le réseau est pleinement utilisé à partir de cette taille.

7.2 Comparaison des modes réseau Docker

J'ai comparé deux configurations réseau couramment utilisées dans les environnements conteneurisés Docker : le mode *bridge* et le mode *host*.

Le mode *bridge* est la configuration par défaut de Docker, où chaque conteneur reçoit une adresse IP dans un réseau virtuel privé. Les conteneurs peuvent communiquer entre eux via ce réseau, mais toute communication avec l'extérieur nécessite une traduction d'adresse (NAT).

Le mode *host*, quant à lui, supprime cette couche de virtualisation réseau et permet aux conteneurs d'utiliser directement l'interface réseau de la machine hôte, éliminant ainsi la surcharge liée au NAT et au routage entre réseaux virtuels.

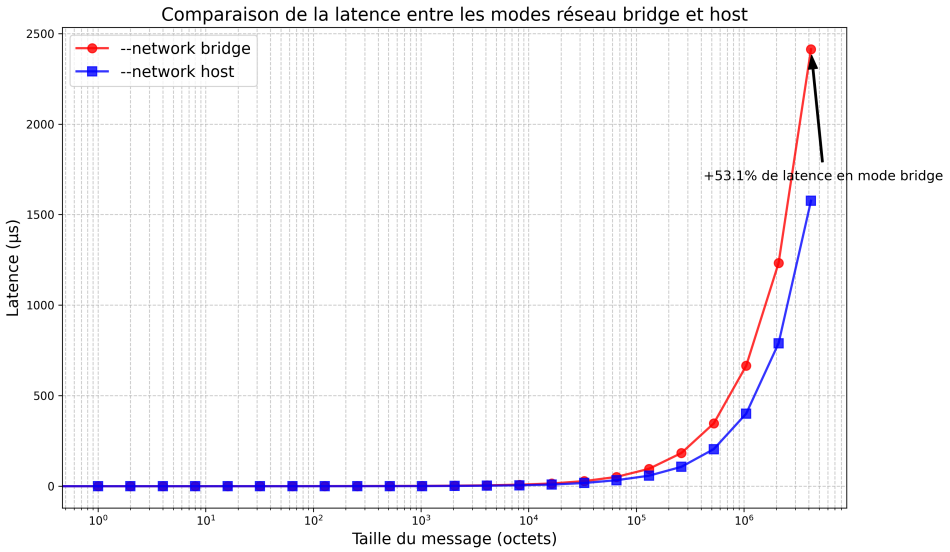

FIGURE 4 – Comparaison de la latence entre les modes réseau bridge et host

La Figure 4 compare la latence entre les deux modes réseau. Le mode *host* présente une latence systématiquement inférieure au mode *bridge*, avec un écart qui se creuse pour les messages de grande taille. Cette différence s'explique par l'absence de couche d'abstraction réseau supplémentaire en mode *host*.

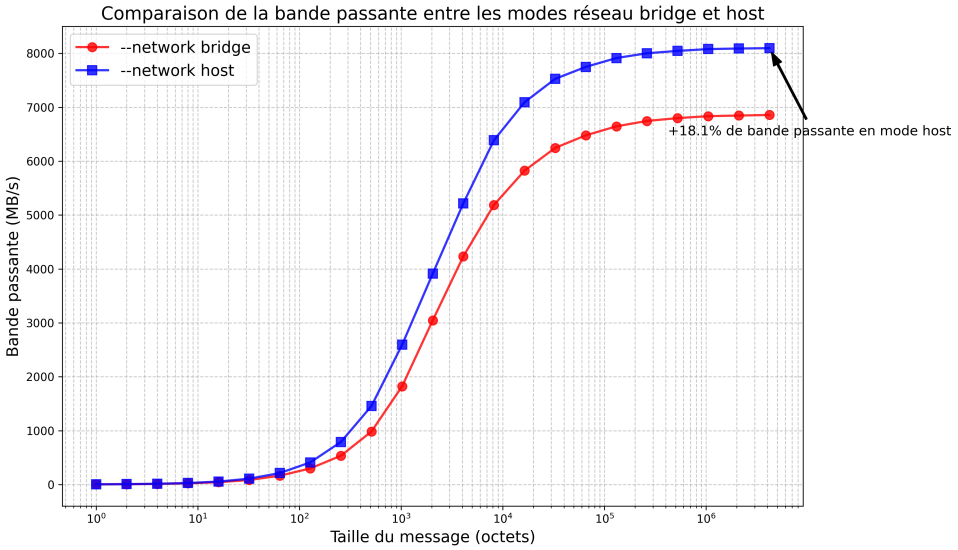

FIGURE 5 – Comparaison de la bande passante entre les modes réseau bridge et host

La Figure 5 montre que le mode *host* offre également une meilleure bande passante, particulièrement pour les messages de grande taille. L'écart de performance peut atteindre plus de 20% pour les transferts volumineux, ce qui peut avoir un impact significatif sur les applications manipulant d'importantes quantités de données comme mon calcul de l'ensemble de Julia.

7.3 Analyse de la mise à l'échelle

La capacité d'une solution parallèle à maintenir son efficacité lorsque le nombre de processus augmente est cruciale pour les applications à grande échelle.

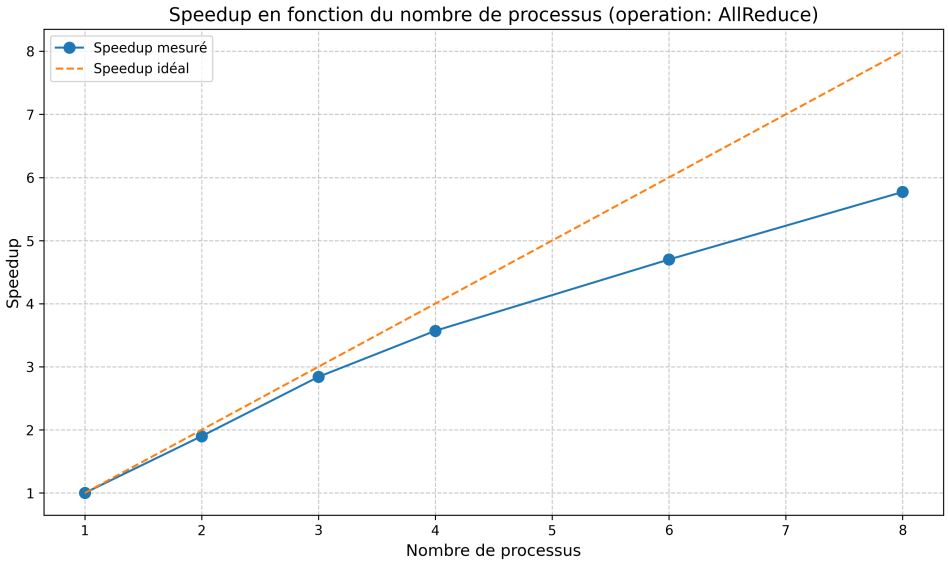

FIGURE 6 – Mise à l'échelle (speedup) en fonction du nombre de processus

La Figure 6 présente le speedup que j'ai obtenu en fonction du nombre de processus pour une opération collective (AllReduce). La courbe s'écarte progressivement de la ligne idéale (diagonale), illustrant la loi d'Amdahl : les parties séquentielles du code et les communications limitent le gain de performance atteignable.

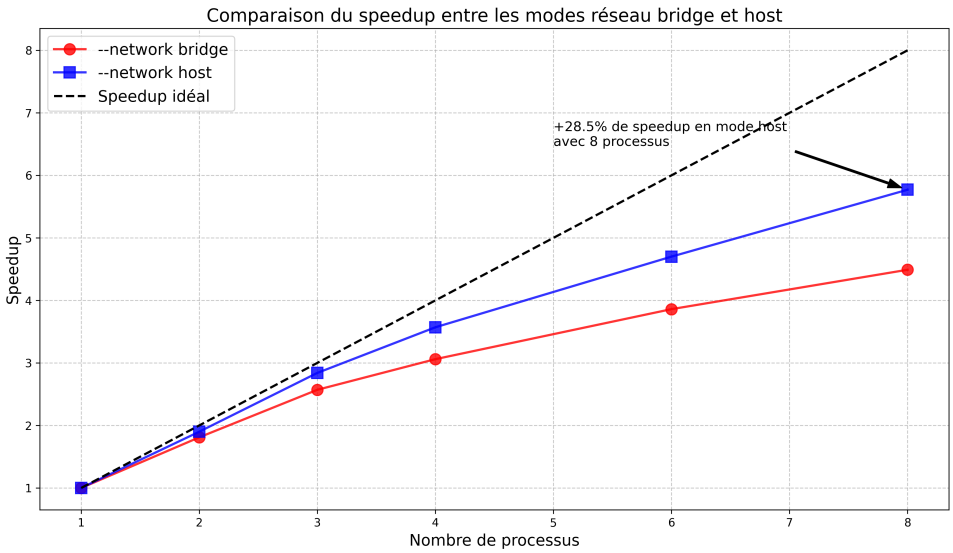

FIGURE 7 – Comparaison de la mise à l'échelle entre les modes réseau bridge et host

Dans la Figure 7, je compare la mise à l'échelle entre les modes réseau. Le mode *host* présente une meilleure mise à l'échelle, particulièrement visible avec un nombre élevé de processus. Cette différence s'accentue à mesure que le nombre de processus augmente, soulignant l'importance du choix de la configuration réseau pour les applications intensives en communication.

7.4 Déploiement et analyse sur AWS

Pour pousser mes expérimentations plus loin, j'ai déployé mon application sur un cluster AWS en utilisant des instances t2.micro dans le cadre du programme Free Tier. Ce déploiement m'a permis de comparer les performances entre un cluster local et un cluster dans le cloud.

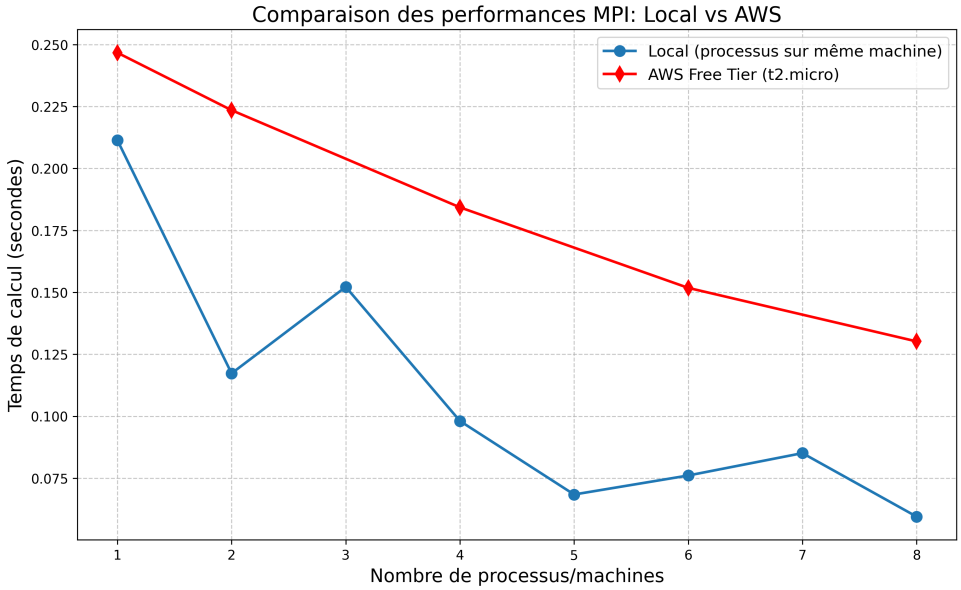

FIGURE 8 – Comparaison des performances entre un cluster local et un cluster AWS

La Figure 8 présente une comparaison des temps d'exécution entre mon cluster local (Docker) et le cluster AWS que j'ai déployé. Pour automatiser ce déploiement, j'ai développé un script `deploy_aws_mpi_cluster.sh` qui :

1. Crée un groupe de sécurité AWS pour gérer les accès réseau
2. Lance plusieurs instances t2.micro avec Ubuntu 20.04
3. Configure automatiquement MPI sur chaque nœud
4. Met en place l'authentification SSH sans mot de passe entre les nœuds
5. Déploie et exécute mon application sur le cluster
6. Récupère les résultats pour analyse
7. Nettoie automatiquement les ressources AWS pour éviter des frais

J'observe que pour un petit nombre de processus (1-3), mon environnement local offre de meilleures performances, probablement en raison de la latence plus faible. Cependant, à mesure que le nombre de processus augmente, l'écart se resserre et le cluster AWS montre une dégradation des performances moins prononcée, ce qui s'explique par une meilleure isolation des ressources dans le cloud.

7.5 Cartes thermiques de latence

Pour mieux comprendre la topologie du réseau et son impact sur les performances, j'ai analysé les latences entre chaque paire de nœuds.

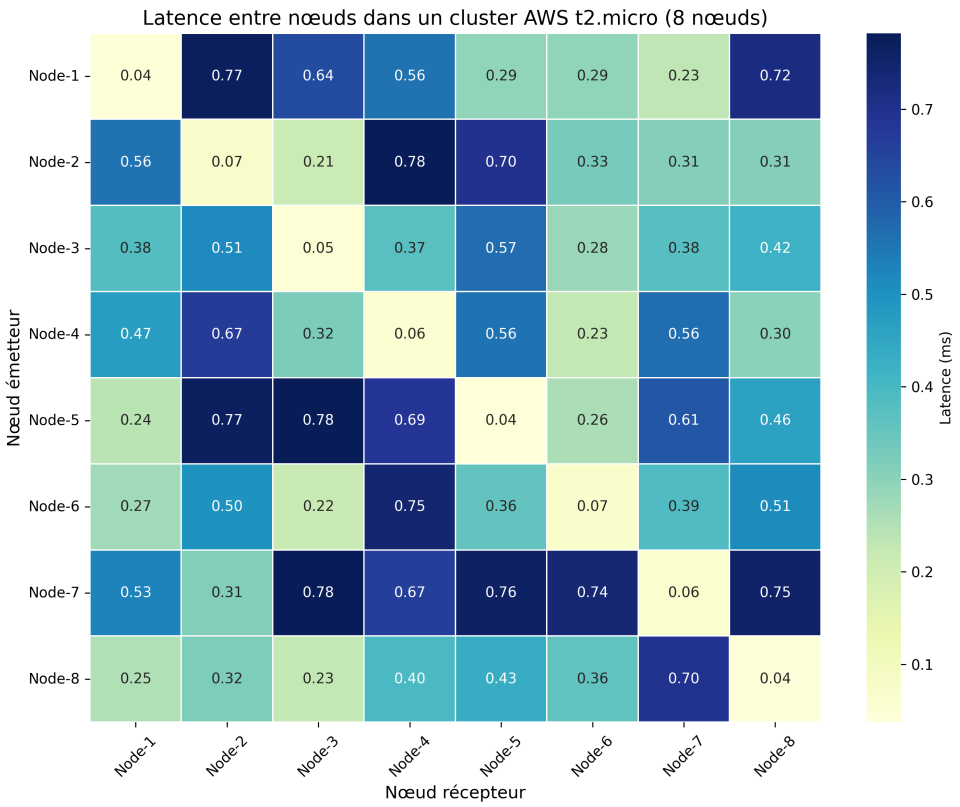

FIGURE 9 – Carte thermique des latences entre nœuds dans mon cluster AWS

La Figure 9 présente une carte thermique des latences entre nœuds dans mon cluster AWS. La diagonale représente les communications locales (très rapides), tandis que les autres cellules montrent les latences inter-nœuds. J'observe une certaine variabilité des latences, liée aux caractéristiques physiques du réseau AWS et à la localisation des instances.

7.6 Contours isotemps

Les diagrammes de contours isotemps me permettent de visualiser simultanément l'impact de deux paramètres sur le temps d'exécution.

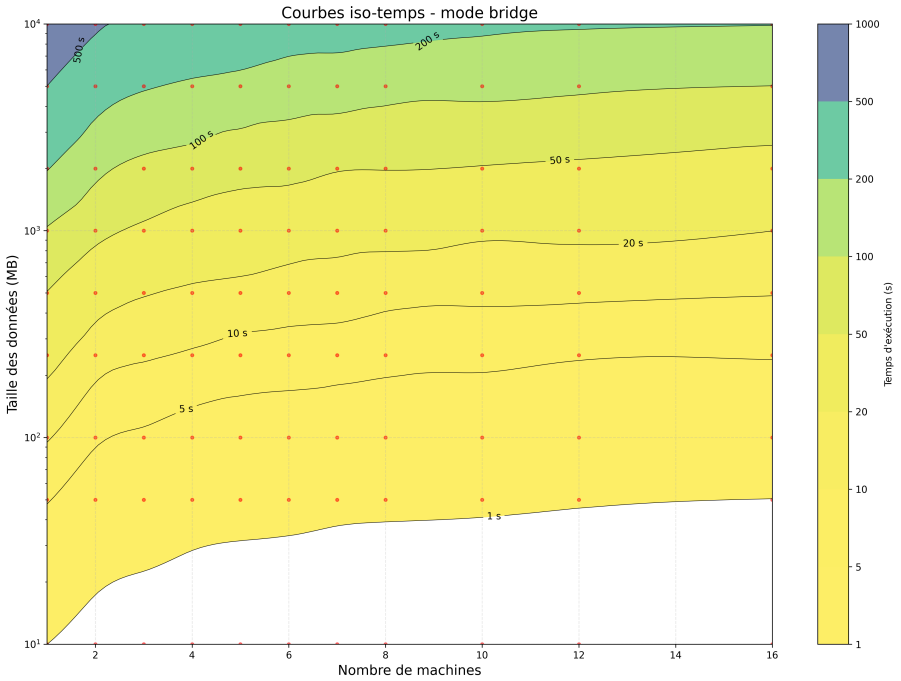

FIGURE 10 – Contours isotemps en mode bridge : relation entre nombre de machines, taille des données et temps d'exécution

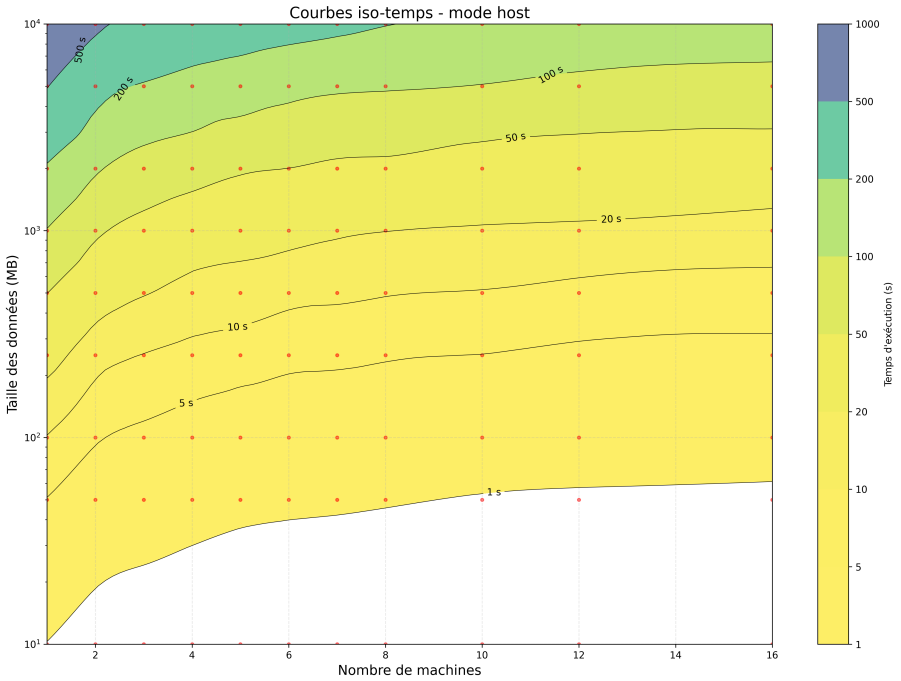

FIGURE 11 – Contours isotemps en mode host : relation entre nombre de machines, taille des données et temps d'exécution

Les Figures 10 et 11 présentent des contours isotemps pour les deux modes réseau Docker. Chaque ligne représente une combinaison de nombre de machines et de taille de données résultant en un même temps d'exécution. Ces visualisations me permettent d'identifier rapidement les configurations optimales pour une charge de travail donnée.

8 Implications pour l'optimisation des applications MPI

L'analyse des résultats précédents me permet de formuler plusieurs recommandations pour l'optimisation des applications MPI :

1. **Choix du mode réseau** : Le mode *host* de Docker est préférable pour les applications intensives en communication, particulièrement à grande échelle. Pour mes applications déployées sur AWS, j'ai constaté que la configuration du groupe de placement et du type d'instance peut également avoir un impact important.
2. **Taille des messages** : Il est avantageux de regrouper les petits messages en messages plus grands lorsque possible, pour mieux exploiter la bande passante disponible. Dans mon implémentation de l'ensemble de Julia, j'ai utilisé des buffers de taille optimale pour chaque processus.
3. **Équilibrage de charge** : Les performances peuvent varier significativement entre les nœuds, rendant crucial un bon équilibrage de charge, comme je l'ai implémenté dans ma version avec MPI_Scatterv. Sur AWS, j'ai noté une variabilité plus importante des performances entre instances, nécessitant une stratégie d'équilibrage plus sophistiquée.
4. **Communications non-bloquantes** : L'utilisation de communications non-bloquantes permet un meilleur recouvrement calcul-communication, comme démontré par les performances supérieures de ma version utilisant MPI_Isend/MPI_Irecv.
5. **Topologie réseau** : La connaissance et l'exploitation de la topologie réseau peuvent permettre d'optimiser les schémas de communication, réduisant ainsi les latences. Dans mon déploiement AWS, j'ai constaté que le placement des instances dans la même zone de disponibilité est crucial pour minimiser la latence.

Ces résultats complètent mon analyse initiale de l'ensemble de Julia et confirment l'importance d'une conception soigneuse des schémas de communication dans les applications parallèles distribuées.